SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 1 – Scenario-Based Requirement Analysis

Course Code:	CSA10 / CSA1010	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Scenario-Based Requirement Analysis
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	S.BHAVYA	**Reg. No.:**	192524376
Date:	22.10.26	**Duration:**	60 minutes

Code of Conduct

I Bhavya S(192524376) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: *Bhavya S*

Annexure A – Scenario-Based Requirement Analysis

Instructions to Students:
Each student will be assigned an individual software project problem statement. Analyze the given scenario and prepare a complete Software Requirement Analysis document by following the steps below.

Question

Based on the **assigned problem statement**, perform a **Scenario-Based Requirement Analysis** and prepare a Software Requirement Specification (SRS) document. Your analysis should include the following:

1. **Study and Analyze the Scenario**
   - Carefully understand the given problem statement.
   - Identify the business domain, stakeholders, existing challenges, and system objectives.
2. **Identify Functional and Non-Functional Requirements**
   - List all functional requirements required by the proposed system.
   - Identify suitable non-functional requirements such as performance, security, reliability, usability, scalability, maintainability, and availability.
3. **Identify Actors and Use Cases**
   - Identify all primary and secondary actors interacting with the system.
   - List the major use cases associated with each actor.
   - Draw a Use Case Diagram representing the interactions between actors and the system.
4. **Prepare the Software Requirement Specification (SRS)**
   Develop an SRS document containing:
   - Introduction
   - Problem Description

- Scope of the System
- Functional Requirements
- Non-Functional Requirements
- Use Case Descriptions
- Data Requirements
- External Interface Requirements
- System Features

5. **Identify Assumptions and Constraints**
   - List the assumptions considered while developing the system.
   - Identify technical, operational, economic, legal, and time-related constraints affecting the project.
6. **Validate Requirement Completeness and Consistency**
   - Verify whether all stakeholder requirements have been addressed.
   - Check for ambiguity, redundancy, inconsistency, and missing requirements.
   - Suggest improvements to enhance the quality and completeness of the requirement specification.

Rubrics for Calculation

Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Scenario Understanding and Problem Analysis (30%)	Demonstrates thorough understanding of the scenario, stakeholders, objectives, and business context with clear analysis.	Demonstrates good understanding with minor omissions.	Basic understanding with limited analysis.	Poor understanding; major aspects missing or incorrect.
Functional and Non-Functional Requirement Identification (25%)	Clearly identifies comprehensive, accurate, and well-classified functional and non-functional requirements.	Identifies most requirements with minor classification errors.	Identifies only basic requirements; several omissions.	Requirements are incomplete, inaccurate, or poorly classified.
Actor Identification and Use Case Modelling (20%)	Correctly identifies all relevant actors and use cases; use case diagram is complete, accurate, and follows UML standards.	Most actors and use cases identified; diagram has minor errors.	Limited actors/use cases identified; diagram partially correct.	Actors/use cases missing or incorrect; diagram absent or inaccurate.
Software Requirement Specification (SRS)	SRS is well-structured, complete, professionally organized, and	SRS is organized with minor missing sections or formatting issues.	SRS includes only essential sections with limited detail.	SRS is incomplete, poorly organized, or

Documentation (15%)	includes all required sections.			lacks essential components.
Assumptions, Constraints, and Requirement Validation (10%)	Clearly identifies realistic assumptions and constraints; validates requirements for completeness, consistency, and ambiguity with appropriate justification.	Identifies most assumptions and constraints; validation is adequate.	Limited assumptions/constraints; validation is superficial.	Assumptions, constraints, and validation are missing or incorrect.

Criteria	**Mark**
Scenario Understanding and Problem Analysis (30%)	/5
Functional and Non-Functional Requirement Identification (25%)	/5
Actor Identification and Use Case Modelling (20%)	/5
Software Requirement Specification (SRS) Documentation (15%)	/5
Assumptions, Constraints, and Requirement Validation (10%)	/5
TOTAL	/25

PERSONAL EXPENSE AND BUDGET MANAGEMENT SYSTEM

1. Study and Analyze the Scenario

Problem Statement: Personal Expense and Budget Management System: Develop an application that allows users to record income and expenses, categorize spending, create budgets, receive alerts, and view financial reports.

1.1 Business Domain

The system belongs to the Personal Finance Management domain. It helps individuals manage their income, monitor expenses, control spending, and plan budgets efficiently.

1.2 Stakeholders

- Primary stakeholder: Registered users who manage personal finances.
- Secondary stakeholders: System administrator, notification service provider, and financial report users.
- Developers and maintainers: Responsible for developing, securing, updating, and maintaining the application.

1.3 Existing Challenges

- Manual recording of income and expenses is time-consuming.
- Users may find it difficult to track spending across different categories.
- Overspending can occur when budgets are not monitored.
- Preparing financial summaries manually may lead to errors.
- Users may miss important budget limits without timely alerts.

1.4 System Objectives

1. Record and maintain income and expense transactions.
2. Categorize spending for easy monitoring.
3. Allow users to create and manage budgets.
4. Send alerts when spending approaches or exceeds budget limits.
5. Generate clear financial reports for a selected period.
6. Protect users' financial information through secure access.

2. Functional and Non-Functional Requirements

2.1 Functional Requirements (FR)

ID	Requirement
FR1	The system shall allow users to register, log in, and log out securely.
FR2	Users shall be able to add, edit, and delete income records.
FR3	Users shall be able to record, update, and delete expenses.
FR4	The system shall categorize expenses, such as food, travel, education, and shopping.
FR5	Users shall be able to create, modify, and delete budgets for a selected period or category.
FR6	The system shall calculate total income, expenses, and remaining balance.
FR7	The system shall monitor expenses against the defined budget.
FR8	The system shall send alerts when spending reaches a configured threshold or exceeds the budget.
FR9	Users shall be able to view financial reports by date, category, and transaction type.
FR10	The system shall allow users to search and filter transactions.
FR11	Users shall be able to view a dashboard containing their financial summary.
FR12	The system shall allow users to manage their profile and notification preferences.

2.2 Non-Functional Requirements (NFR)

Type	Requirement
Performance	Common operations and reports should respond within 3 seconds under normal load.
Security	Passwords must be securely hashed, and users must access only their own financial records.
Reliability	Transactions must be stored accurately without unintended data loss.
Usability	The interface should be simple, clear, and easy for beginners to use.
Scalability	The system should support increasing numbers of users and transactions.
Maintainability	The application should use modular code for easier updates and debugging.
Availability	The application should be accessible whenever the service is operational, with planned maintenance communicated.
Data integrity	Income, expenses, categories, and budgets must be validated before saving.

3. Actor Identification and Use Case Modelling

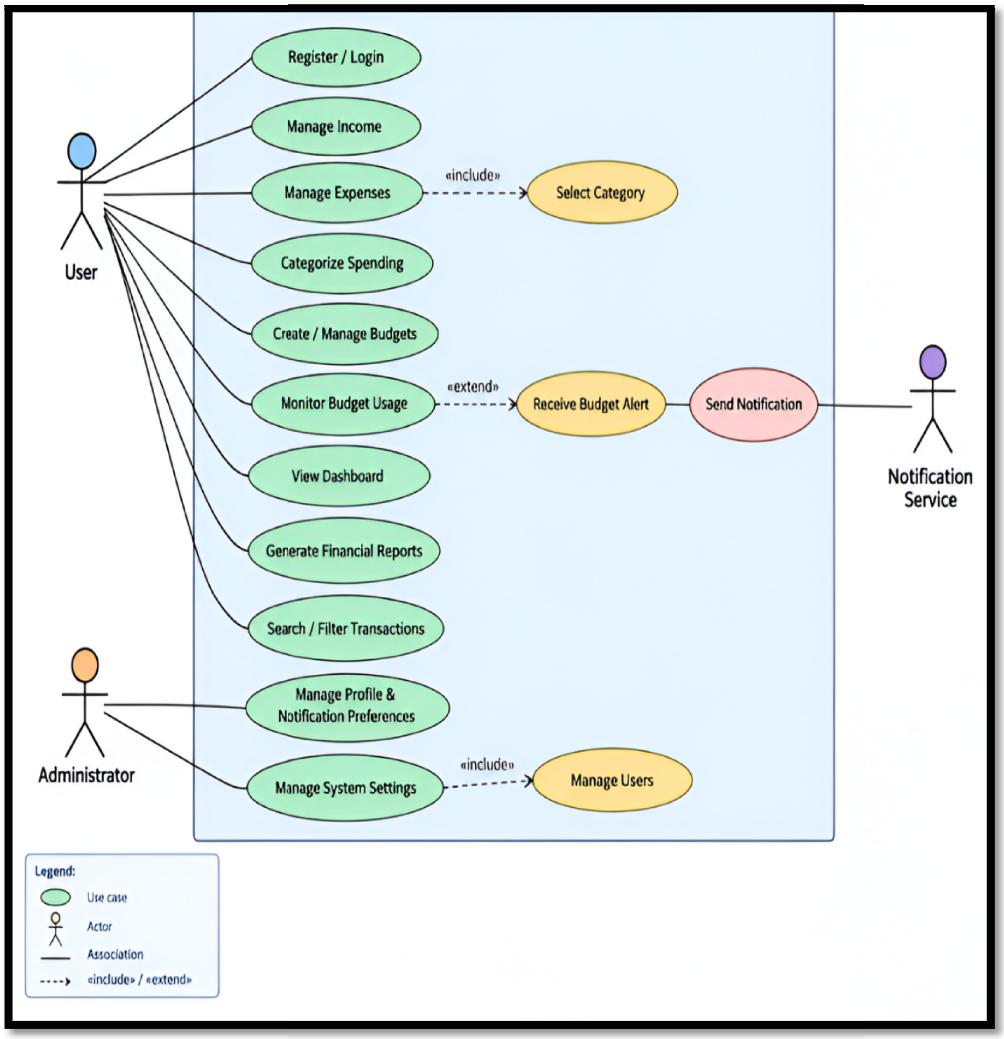
3.1 Actors

Actor	Role
User	Records income and expenses, manages budgets, views reports, and receives alerts.
Administrator	Manages system settings, user accounts, and application maintenance.
Notification Service	Delivers budget alerts through supported notification channels.

3.2 Major Use Cases

- Register and log in
- Manage income
- Manage expenses
- Categorize spending
- Create and manage budgets
- Monitor budget usage
- Receive budget alerts
- View dashboard and financial reports
- Search and filter transactions
- Manage profile and system settings

3.3 Use Case Diagram



4. Software Requirement Specification (SRS)

4.1 Introduction

The Personal Expense and Budget Management System is a software application designed to help individuals manage their finances. It provides transaction recording, expense categorization, budget planning, automatic calculations, alerts, and financial reporting.

4.2 Problem Description

Traditional expense tracking methods require manual calculations and make it difficult to identify spending patterns. Without proper budget monitoring, users may overspend and fail to plan their finances effectively. The proposed system provides a centralized and automated solution to overcome these problems.

4.3 Scope of the System

The system will:

- Maintain personal income and expense records.
- Organize transactions by category and date.
- Create monthly or category-based budgets.
- Calculate balances and monitor spending.
- Generate financial summaries and reports.
- Notify users about budget thresholds.

The initial version focuses on individual financial management. Direct bank integration, automatic bank transaction imports, and investment management are outside the initial scope.

4.4 Functional Requirements

The system shall provide user authentication, transaction management, category management, budget creation, budget monitoring, alerts, transaction filtering, dashboard summaries, financial reports, and profile settings. Each user shall have access only to their own financial information.

4.5 Non-Functional Requirements

The system shall be secure, reliable, responsive, maintainable, usable, and scalable. It shall validate inputs, preserve transaction accuracy, restrict unauthorized access, and support future feature enhancements.

4.6 Use Case Descriptions

Use case	Actor	Description
Login	User	Enters valid credentials to access the application.
Add Expense	User	Enters amount, date, category, and description to record an expense.
Add Income	User	Records income details and updates the financial summary.
Create Budget	User	Defines a spending limit for a period or category.
Receive Alert	User, Notification Service	Receives a notification when spending reaches a configured threshold or exceeds the budget.
Generate Report	User	Selects a period and views income, expenses, and spending summaries.
Manage Users	Administrator	Performs authorized account and system-management activities.

4.7 Data Requirements

The system shall store the following information:

Entity	Main attributes
User	User ID, name, email, password hash
Income	Income ID, user ID, amount, date, source
Expense	Expense ID, user ID, amount, date, category ID, description
Category	Category ID, category name, category type
Budget	Budget ID, user ID, category ID, budget amount, start date, end date
Alert	Alert ID, user ID, budget ID, threshold, message, timestamp
Report	Report ID, user ID, report period, income total, expense total, balance

Data validation: Amounts must be valid positive values, required fields cannot be empty, dates must be valid, and each transaction must be linked to the correct user. Budget periods must be valid and consistent.

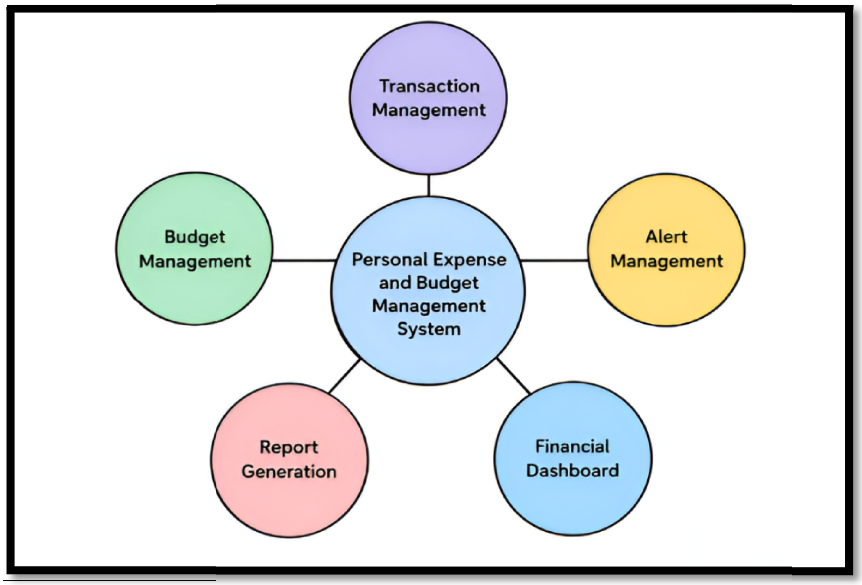
4.8 External Interface Requirements

- User interface: Login page, transaction forms, budget page, dashboard, reports, and settings.
- Software interface: Application backend, database management system, and reporting component.
- Notification interface: Email, push notification, or another supported notification service.
- Hardware interface: Computer or smartphone with a supported browser or application environment.
- Communication interface: Secure HTTPS communication for a deployed web application.

4.9 System Features

1. Transaction Management: Add, edit, delete, and search income and expense records.
2. Budget Management: Set spending limits and monitor budget usage.
3. Alert Management: Notify users when spending reaches configured limits.
4. Financial Dashboard: Display total income, expenses, remaining balance, and category-wise spending.
5. Report Generation: Produce date-wise and category-wise summaries for financial planning.

.



5.

Assumptions and Constraints

5.1 Assumptions

1. Users have a valid email address or supported login credentials.
2. Users enter correct and up-to-date income and expense details.
3. Users define their own budget amounts and alert thresholds.
4. Internet access is available for the deployed web application and online notifications.
5. Users have permission to access only their own financial records.

5.2 Constraints

Constraint type	Description
Technical	The system requires a compatible application environment and database.
Operational	Users must record transactions regularly to obtain accurate reports.
Economic	Development and hosting costs must remain within the available budget.
Legal	Personal and financial information must be handled according to applicable privacy and data-protection laws.
Time	Development, testing, and deployment must be completed within the project deadline.

6. Requirement Completeness and Consistency Validation

6.1 Validation Checklist

Requirement validation

- ✓ All scenario objectives are covered by functional requirements.
- ✓ Every major use case has a corresponding actor.
- ✓ Income, expense, budget, and alert data are defined.
- ✓ Security and access control requirements are specified.
- ✓ Budget alerts and report generation are included.
- ✓ No contradictory or duplicate requirements remain.
- ✓ Performance requirements are measurable and testable.

6.2 Validation Findings

- Completeness: All major functions specified in the scenario are included.
- Consistency: Functional requirements align with the identified actors, use cases, and data entities.
- Ambiguity: Terms such as “budget alert” should have a configurable numerical threshold.
- Redundancy: Transaction management and report generation should remain separate but connected functions.
- Improvement: Add transaction backup and recovery, currency support, and export options in future versions.

6.3 Sample Requirement Test Cases

Test case	Input or action	Expected result
TC1	Add a valid expense	Expense is saved and totals are updated.
TC2	Enter an invalid expense amount	Validation error is displayed.
TC3	Set a budget and reach its alert threshold	Budget notification is triggered.
TC4	Generate a monthly report	Correct income, expenses, and balance are displayed.
TC5	Attempt to access another user's records	Access is denied.

7. Conclusion

The Personal Expense and Budget Management System provides an organized and efficient way to manage personal finances. The requirements analysis identifies the system objectives, functional and non-functional requirements, actors, use cases, data needs, assumptions, and constraints. Requirement validation helps ensure that the proposed system is complete, consistent, secure, and suitable for implementation.